



Taller 2 – Taller Defect Density

Programa: Ingeniería De Sistemas

Asignatura: Calidad De Software

Profesor: José Miguel llanos Mosquera

Integrantes

María Sofia Aljure Herrera

María Juliana Ferro Bonilla

José Miguel Vera Garzón

Corporación Universitaria Del Huila Corhuila

Neiva – Huila

Marzo 2026



Contenido	
Introducción	3
Marco teórico	3
Calidad de software	3
SonarQube	4
Defect Density	4
Clasificación de problemas en SonarQube	5
Importancia del análisis estático	6
Objetivos	6
Objetivo general	6
Objetivos específicos	6
Herramientas utilizadas	7
IntelliJ IDEA	7
Java JDK 17	7
Maven	7
SonarQube	7
Docker	7
JaCoCo	8
Desarrollo de las tareas	8
Identificación de issues en SonarQube	8
Clasificación de issues (High, Medium, Low)	9
Cálculo del Defect Density	10
Fórmula utilizada	10
Desarrollo	10
Interpretación	10
Cálculo	10
Resultado	11
Interpretación	11
Análisis	11
Issue High	12
Issue Medium	15
Issue Low	27
Conclusiones	37



Introducción

La calidad del software constituye un pilar fundamental en el ciclo de vida del desarrollo de aplicaciones, dado que determina en gran medida la capacidad de un sistema para satisfacer requerimientos de mantenibilidad, seguridad, confiabilidad y eficiencia operativa. En este escenario, el análisis estático de código emerge como una práctica esencial para la detección temprana de defectos, permitiendo intervenir antes de que estos comprometan entornos productivos.

El presente documento expone los resultados de un ejercicio práctico orientado a la aplicación de la métrica Defect Density, utilizando como stack tecnológico SonarQube, Docker, Java e IntelliJ IDEA. Como objeto de estudio se seleccionó un proyecto académico con defectos sembrados de manera intencional, lo cual facilitó la identificación sistemática de issues asociados a errores de programación, vulnerabilidades de seguridad y desviaciones de buenas prácticas de codificación.

Para cada hallazgo reportado por SonarQube se diseñaron e implementaron estrategias de corrección específicas, con el propósito de elevar la calidad interna del código base. El proceso concluyó con una nueva ejecución del análisis estático, comparando el estado inicial contra el estado final del proyecto, lo que permitió cuantificar el impacto de las mejoras aplicadas y la reducción efectiva de la densidad de defectos.

Marco teórico

Calidad de software



La calidad de software puede entenderse como el nivel en que un programa o sistema cumple con lo que se espera de él, tanto en su funcionamiento como en su estructura interna. No se trata únicamente de que el software “sirva”, sino de que también sea confiable, seguro, fácil de mantener y eficiente.

Un software con buena calidad permite que los usuarios realicen sus tareas correctamente, pero además facilita el trabajo de los desarrolladores cuando se necesita corregir errores, agregar nuevas funciones o mejorar el rendimiento. Por esta razón, evaluar la calidad del código es una actividad importante dentro del desarrollo de software.

SonarQube

SonarQube es una herramienta de análisis estático de código que ayuda a identificar problemas dentro de un proyecto de software sin necesidad de ejecutar el programa. A través de esta plataforma es posible detectar errores, malas prácticas de programación, posibles vulnerabilidades de seguridad y aspectos que afectan la mantenibilidad del código.

Su utilidad principal radica en que permite visualizar el estado de calidad de un proyecto de forma organizada, mostrando métricas, reportes e issues que orientan al desarrollador sobre qué debe corregir. En este taller, SonarQube fue fundamental para encontrar los problemas presentes en el proyecto Java y analizar su impacto en la calidad del software.

Defect Density

La **Defect Density** o densidad de defectos es una métrica utilizada para medir la cantidad de defectos encontrados en un sistema de software con relación al tamaño del código fuente. Generalmente, este tamaño se expresa en miles de líneas de código, conocidas como **KLOC**.



La fórmula utilizada es la siguiente:

$$DD = \text{Número de defectos} / KLOC$$

Donde:

$$KLOC = \text{Líneas de código} / 1000$$

Esta métrica es útil porque permite tener una idea más clara de la calidad del software.

Un valor alto indica que existen muchos defectos en relación con el tamaño del sistema, mientras que un valor bajo sugiere una mejor calidad. En el desarrollo de este taller se toma como meta un valor inferior a **5 defectos por KLOC**, ya que este rango representa un nivel aceptable de calidad.

Clasificación de problemas en SonarQube

En SonarQube, los problemas detectados en el código pueden aparecer clasificados por nivel de severidad. En este taller, dichos problemas se identifican principalmente como **High**, **Medium** y **Low**, lo cual permite reconocer cuáles requieren atención inmediata y cuáles representan un impacto menor.

Los issues de nivel High corresponden a problemas de mayor gravedad, ya que pueden comprometer la seguridad, generar fallos importantes o afectar significativamente la lógica del programa. Los issues Medium representan problemas de impacto intermedio, relacionados muchas veces con errores de implementación, validaciones insuficientes o aspectos que dificultan el mantenimiento del código. Finalmente, los issues Low suelen estar asociados con detalles menores, como variables no utilizadas, código redundante o prácticas que no son críticas, pero que sí disminuyen la limpieza y claridad del proyecto.



Esta clasificación resulta útil porque ayuda a priorizar las correcciones y a enfocar primero los esfuerzos en los problemas más importantes.

Importancia del análisis estático

El análisis estático de código es una práctica que permite revisar un programa sin necesidad de ejecutarlo. Su importancia está en que ayuda a detectar errores desde etapas tempranas del desarrollo, evitando que estos lleguen a producción o generen consecuencias más graves en el futuro.

Además, este tipo de análisis favorece la aplicación de buenas prácticas de programación, mejora la legibilidad del código y reduce el esfuerzo necesario para mantener el sistema. En el contexto académico, también permite comprender de manera más clara cómo pequeños errores o malas decisiones en el código pueden afectar la calidad general de un proyecto.

Objetivos

Objetivo general

Analizar la calidad del proyecto Java propuesto mediante SonarQube para identificar defectos, calcular la métrica Defect Density, implementar mejoras en el código fuente y comparar los resultados antes y después de la corrección de issues.

Objetivos específicos

1. Configurar el entorno de trabajo con Docker, SonarQube, Java, Maven e IntelliJ IDEA para ejecutar el análisis estático del proyecto.
2. Identificar y clasificar los issues detectados por SonarQube, describiendo el problema asociado a cada uno.



3. Proponer e implementar alternativas de solución basadas en buenas prácticas, código limpio y mejora de mantenibilidad (patrones y antipatrones).
4. Reejecutar el análisis de SonarQube para comparar los resultados obtenidos y evaluar la variación de la densidad de defectos.

Herramientas utilizadas

- **IntelliJ IDEA**

Entorno de desarrollo integrado utilizado para crear, editar, compilar y ejecutar el proyecto Java.

- **Java JDK 17**

Plataforma de desarrollo empleada para compilar y ejecutar el código fuente del proyecto.

- **Maven**

Herramienta de automatización y gestión de dependencias utilizada para compilar el proyecto, ejecutar pruebas y generar reportes de cobertura.

- **SonarQube**

Plataforma de análisis estático usada para detectar bugs, vulnerabilidades y code smells, además de visualizar métricas de calidad.

- **Docker**

Tecnología de contenedores utilizada para desplegar SonarQube de forma local y facilitar su ejecución sin configuraciones complejas.



- **JaCoCo**

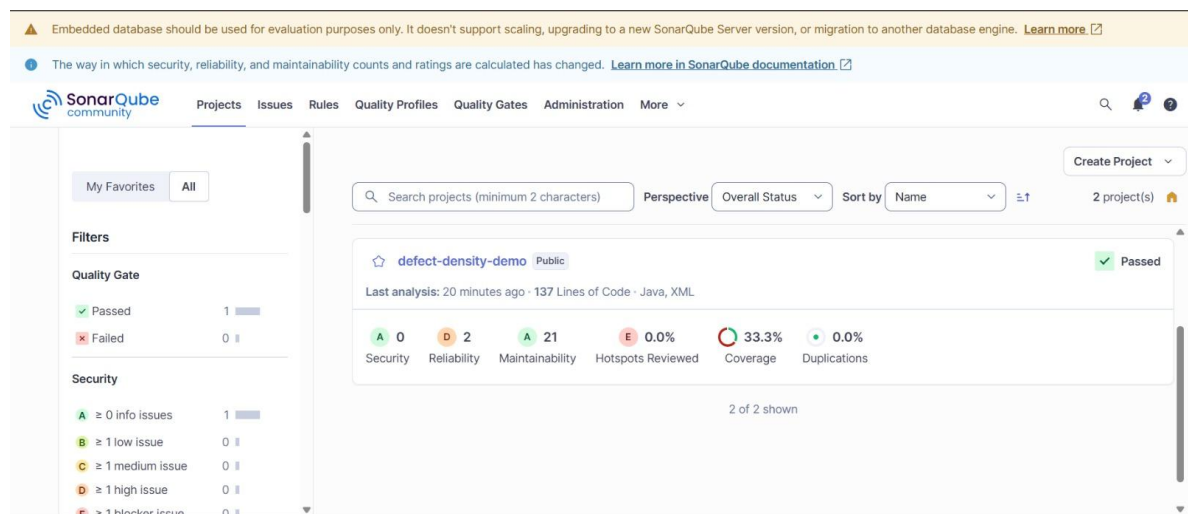
Plugin empleado para medir la cobertura de pruebas del proyecto Java.

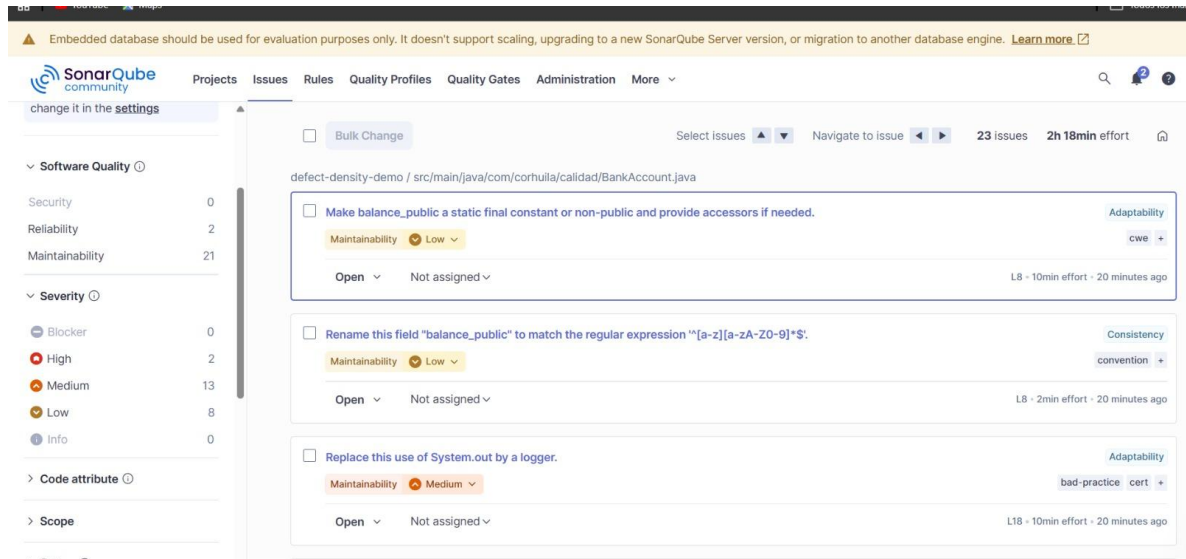
Desarrollo de las tareas

Identificación de issues en SonarQube

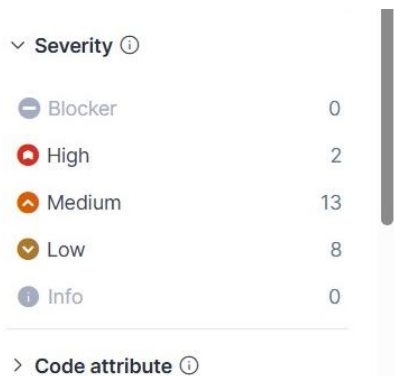
Una vez ejecutado el análisis del proyecto Java en la plataforma SonarQube, se identificaron un total de **23 issues** en el código fuente. Estos problemas corresponden a diferentes tipos de fallos relacionados con seguridad, errores de programación y malas prácticas de desarrollo.

Los issues detectados evidencian la presencia de vulnerabilidades, problemas de mantenibilidad y deficiencias en la estructura del código, lo cual impacta directamente la calidad del software.





Clasificación de issues (High, Medium, Low)



Los issues identificados fueron clasificados según su nivel de severidad, permitiendo priorizar su corrección de la siguiente manera:

Severidad	Cantidad
High	2
Medium	13
Low	8
Total	23



Los issues de tipo **High** representan problemas críticos que pueden comprometer la seguridad o el correcto funcionamiento del sistema.

Los issues **Medium** corresponden a problemas que afectan la lógica del código o su mantenibilidad, mientras que los **Low** están relacionados con aspectos menores como buenas prácticas, código innecesario o detalles de estilo.

Cálculo del Defect Density

Fórmula utilizada

DD = Número de defectos / KLOC

KLOC = Líneas de código / 1000

Desarrollo

Total, de issues: **23**

Líneas de código (LOC): **63**

Interpretación

Si $DD < 5 \rightarrow$ calidad alta

Si DD entre 5 y 10 $\rightarrow$ calidad media

Si $DD > 10 \rightarrow$ calidad baja

Cálculo

Primero se calcula el tamaño del sistema en KLOC:

$KLOC = 63 / 1000 = 0.063$



Luego se aplica la fórmula de **Defect Density**:

$$DD = 23 / 0.063 \approx 365.08 \text{ defectos/KLOC}$$

Resultado

$$\text{Defect Density} \approx 365 \text{ defectos/KLOC}$$

Interpretación

El valor obtenido de la densidad de defectos es extremadamente alto, ya que supera ampliamente el valor recomendado de menos de 5 defectos/KLOC.

Esto indica que el proyecto presenta una baja calidad de software, con una alta cantidad de problemas en relación con su tamaño. Los resultados evidencian la necesidad de realizar actividades de refactorización, corrección de errores y aplicación de buenas prácticas de programación.

Análisis

La alta densidad de defectos se debe principalmente a:

- Presencia de vulnerabilidades de seguridad
- Uso de malas prácticas de programación
- Código innecesario o no utilizado
- Falta de validaciones
- Problemas de lógica en algunos métodos

Esto confirma la importancia de utilizar herramientas como SonarQube para detectar estos problemas en etapas tempranas del desarrollo.

ISSUE HIGH

HIGH 1

```
21 // METODO 3: Complejidad cognitiva muy alta
22 public String processUser(String user) {
23
24     if (user != null) {
25         if (user.length() > 0) {
26             if (user.contains("admin")) {
27                 if (user.contains("test")) {
28                     if (user.contains("prod")) {
29
30                         return "ADMIN_TEST_PROD";
31                     } else {
32                         return "ADMIN_TEST";
33                     }
34                 } else {
35                     return "ADMIN";
36                 }
37             } else {
38                 return "USER";
39             }
40         }
41     }
42 }
```

Refactor this method to reduce its Cognitive Complexity from 18 to the 15 allowed.

El método processUser tiene múltiples estructuras if anidadas, lo que incrementa la complejidad cognitiva a 18. Esto dificulta la lectura, mantenimiento y comprensión del código, ya que obliga a seguir varias condiciones encadenadas. Además, no se aplican buenas prácticas



como early return o simplificación de condiciones. Este tipo de código es propenso a errores y difícil de escalar si se agregan nuevas reglas.

Anti-patrón:

Se identifica el anti-patrón Arrow Code, caracterizado por múltiples estructuras condicionales anidadas que incrementan la complejidad cognitiva y dificultan la legibilidad del código.

Patrón aplicado (Dos alternativas):

1. Se implementa el patrón Guard Clauses, el cual reduce la anidación mediante validaciones tempranas y retornos anticipados, mejorando la claridad, mantenibilidad y estructura del código.

2. También podría aplicarse el patrón Strategy, ya que el método toma decisiones basadas en múltiples condiciones. Este patrón permitiría encapsular cada comportamiento en clases independientes. Sin embargo, para este caso específico, su uso se considera innecesario debido a la simplicidad de la lógica, por lo que se opta por la implementación de Guard Clauses, que ofrece una solución más clara y ligera.

HIGH 2

```
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception {
56     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
    Use try-with-resources or close this "Stream" in a "finally" clause.
57     .forEach(System.out::println);
58     return "File read"; // Stream no se cierra explícitamente
59 }
60
61 public List<String> getUsers() {
62     return users;
63 }
64 }
```

El método `readUserFile` abre un `Stream` mediante `Files.lines()` pero no lo cierra explícitamente. Esto genera un `resource leak`, ya que el flujo queda abierto y puede consumir memoria o descriptores del sistema. En aplicaciones grandes o con múltiples lecturas, esto puede provocar degradación del rendimiento o incluso fallos. El problema ocurre porque no se utiliza un mecanismo adecuado de liberación de recursos como `try-with-resources`.

Anti-patrón identificado:

Resource Leak (Fuga de recursos)

Se adquiere un recurso (`Stream` de archivo) pero no se libera. Esto viola buenas prácticas de manejo de recursos en Java.

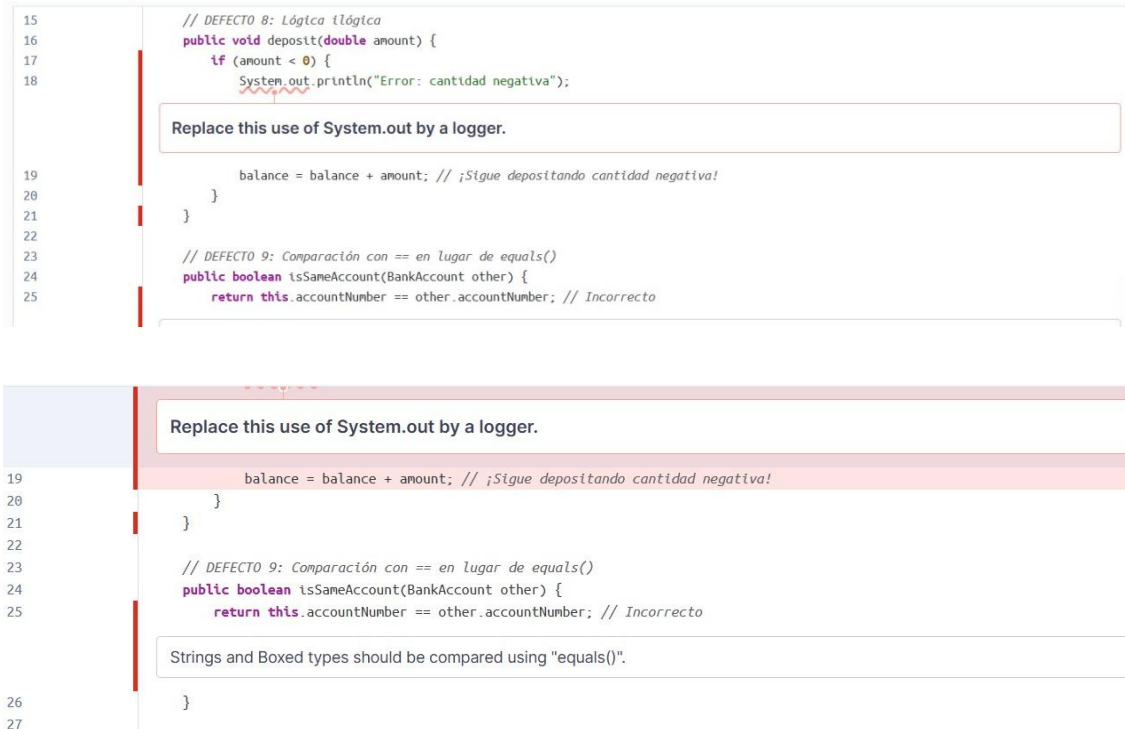
Patrón aplicado:

Try-With-Resources: Este patrón cierra automáticamente los recursos, evita fugas de memoria y hace el código más seguro y limpio.



ISSUE MEDIUM

MEDIUM 1



Este código presenta problemas de validación, uso incorrecto de comparación de objetos y malas prácticas de salida por consola. La lógica permite depósitos negativos, lo cual es un error crítico de negocio. Además, usar == en objetos puede generar fallos sutiles. La solución mejora la seguridad y confiabilidad aplicando validaciones tempranas (Guard Clauses), uso de equals() y reemplazo de System.out por logging profesional. Esto hace el código más robusto, mantenible y alineado con buenas prácticas.

Anti-patrón:

Se identifican múltiples problemas: uso de impresión directa en consola (System.out), lógica defectuosa en validaciones y comparación incorrecta de objetos con ==. Esto refleja malas prácticas de control de flujo y manejo de objetos.

Patrón aplicado (Dos alternativas):

- Se implementa el patrón Guard Clauses, validando condiciones inválidas desde el inicio y evitando ejecutar lógica incorrecta. Además, se reemplaza System.out por un logger y se usa equals() para comparación correcta.
- También podría aplicarse el patrón Command, encapsulando operaciones como depósito en objetos independientes. Sin embargo, para este caso resulta innecesario, por lo que Guard Clauses es más simple y adecuado.

MEDIUM 2

```
23 // DEFECTO 9: Comparación con == en lugar de equals()
24 public boolean tsSameAccount(BankAccount other) {
25     return this.accountNumber == other.accountNumber; // Incorrecto
26 }
27
28 // DEFECTO 10: Código muerto
29 public double getBalance() {
30     double temp = balance * 2;
31     return balance; // temp nunca se usa
```

Strings and Boxed types should be compared using "equals()".

Remove this useless assignment to local variable "temp".

El código contiene errores comunes como comparación incorrecta de objetos y presencia de código muerto, lo cual afecta la calidad y claridad del sistema. Estas malas prácticas pueden generar confusión y dificultar el mantenimiento. La solución elimina variables innecesarias y corrige la comparación usando equals(). Esto mejora la legibilidad, reduce posibles bugs y sigue principios de Clean Code, manteniendo el código más limpio, eficiente y comprensible.

Anti-patrón:

Se identifica comparación incorrecta con == y presencia de código muerto (variable no utilizada).

Patrón aplicado (Dos alternativas):

- Se aplica el principio de Clean Code (eliminación de código muerto) junto con el uso
- Alternativamente, se podría aplicar Refactoring (Extract Method) si existieran más cálculos, pero en este caso no es necesario.

MEDIUM 3

```
25         return this.accountNumber == other.accountNumber; // Incorrecto
26     }
27
28     // DEFECTO 10: Código muerto
29     public double getBalance() {
30         double temp = balance * 2;
31
32         return balance; // temp nunca se usa
33     }
```

Strings and Boxed types should be compared using "equals()".

Remove this useless assignment to local variable "temp".

Se repiten errores del código anterior, lo que indica falta de refactorización y control de calidad. La duplicación de malas prácticas aumenta el riesgo de inconsistencias en el sistema. Se corrige eliminando redundancias y aplicando buenas prácticas como el uso de equals() y eliminación de código innecesario. Esto permite tener un código más coherente, fácil de mantener y menos propenso a errores en el futuro.

Anti-patrón:

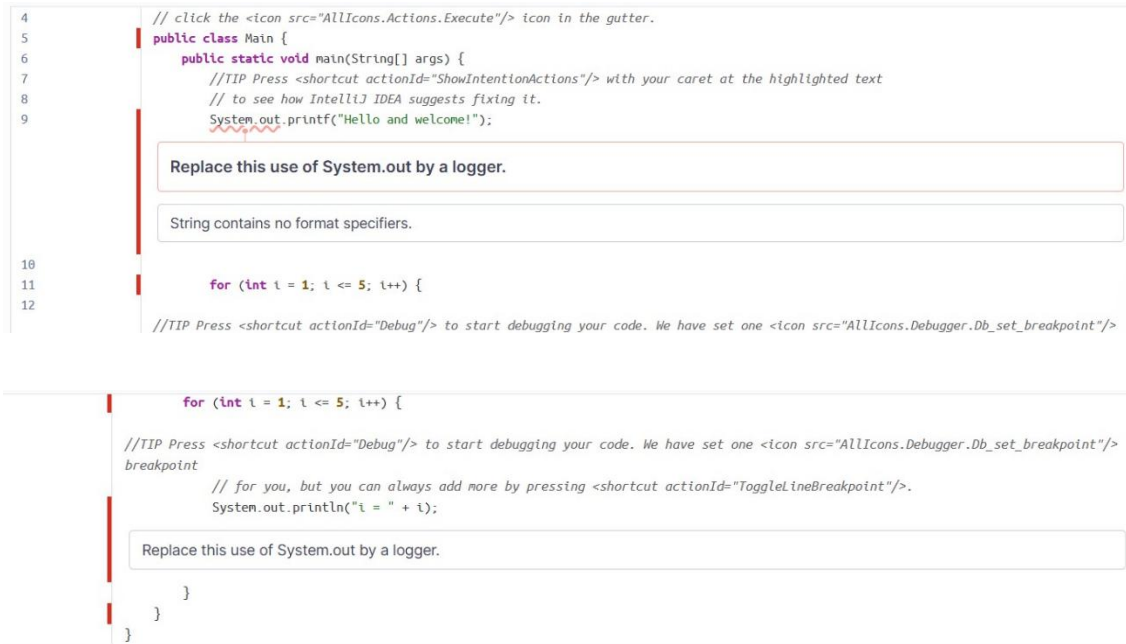
Duplicación de errores: comparación incorrecta con == y código muerto.

Patrón aplicado (Dos alternativas):

- Aplicación de Refactoring básico + Clean Code eliminando redundancias.

- Se podría aplicar Value Object Pattern para encapsular accountNumber, pero sería excesivo aquí.

MEDIUM 4



```
4 // click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.
5 public class Main {
6     public static void main(String[] args) {
7         //TIP Press <shortcut actionId="ShowIntentionActions"/> with your caret at the highlighted text
8         // to see how IntelliJ IDEA suggests fixing it.
9         System.out.printf("Hello and welcome!");
10
11         for (int i = 1; i <= 5; i++) {
12
13             //TIP Press <shortcut actionId="Debug"/> to start debugging your code. We have set one <icon src="AllIcons.Debugger.Db_set_breakpoint"/>
14             // breakpoint
15             // for you, but you can always add more by pressing <shortcut actionId="ToggleLineBreakpoint"/>.
16             System.out.println("i = " + i);
17
18         }
19     }
20 }
```

Replace this use of System.out by a logger.

String contains no format specifiers.

El uso de System.out para salida de información no es adecuado en aplicaciones reales, especialmente en entornos productivos. Además, printf está mal utilizado sin formato. Se reemplaza por un sistema de logging, lo cual permite mejor control, niveles de registro y trazabilidad. Esta mejora facilita la depuración y el monitoreo del sistema, alineándose con estándares profesionales de desarrollo.

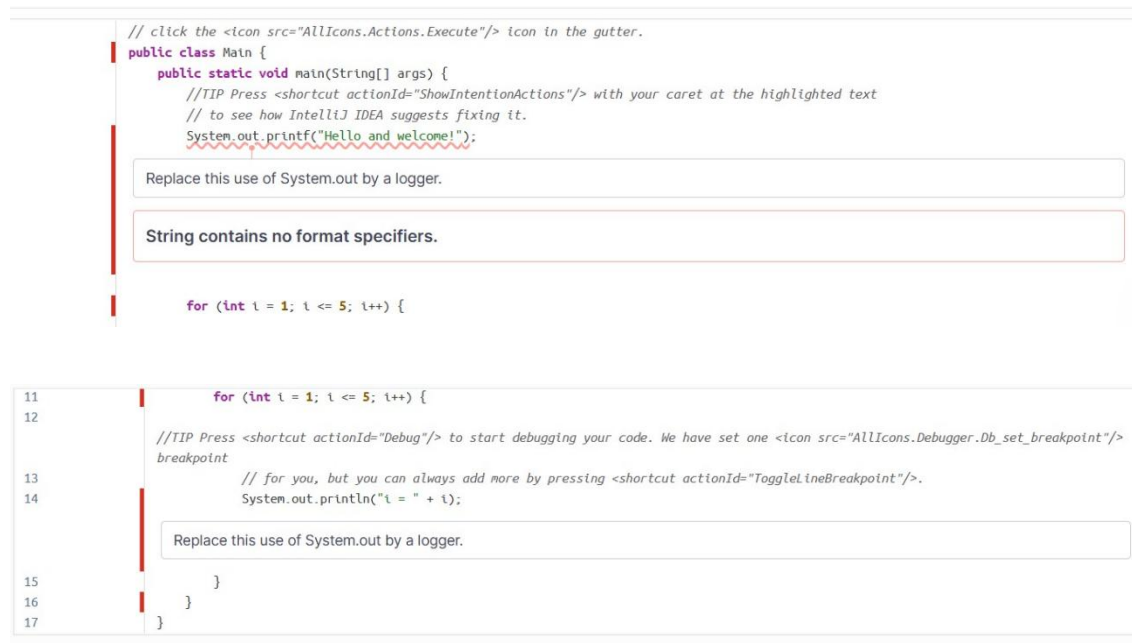
Anti-patrón:

Uso de System.out en lugar de logging y uso incorrecto de printf sin formato.

Patrón aplicado (Dos alternativas):

- Se aplica el patrón Logging Strategy, reemplazando salidas por consola con un logger adecuado.
- Alternativamente, se podría usar un framework como Log4j (Strategy de logging configurable), pero para este caso el logger básico es suficiente.

MEDIUM 5



Este caso repite el problema de uso de consola para mostrar información. Aunque funcional, no es escalable ni adecuado para sistemas reales. El uso de logging permite controlar mejor la salida, registrar eventos importantes y facilitar la auditoría. La solución mejora la calidad del código sin aumentar complejidad, aplicando buenas prácticas estándar en desarrollo de software.

Anti-patrón:

Duplicación del mismo problema: uso de System.out y mal uso de printf.

Patrón aplicado (Dos alternativas):

- Aplicación de Logging Pattern.
- Se podría aplicar Observer Pattern para eventos de logging, pero es innecesario aquí.

MEDIUM 6



```
9      System.out.printf("Hello and welcome!");

10
11
12      for (int i = 1; i <= 5; i++) {
13          //TIP Press <shortcut actionId="Debug"/> to start debugging your code. We have set one <icon src="AllIcons.Debugger.Db_set_breakpoint"/>
14          // breakpoint
15          // for you, but you can always add more by pressing <shortcut actionId="ToggleLineBreakpoint"/>.
16          System.out.printf("i = " + i);
17      }
```



```
10
11
12      for (int i = 1; i <= 5; i++) {
13          //TIP Press <shortcut actionId="Debug"/> to start debugging your code. We have set one <icon src="AllIcons.Debugger.Db_set_breakpoint"/>
14          // breakpoint
15          // for you, but you can always add more by pressing <shortcut actionId="ToggleLineBreakpoint"/>.
16          System.out.printf("i = " + i);
17      }
```

Se evidencia nuevamente el uso incorrecto de impresión por consola. Aunque útil en pruebas rápidas, no es recomendable en aplicaciones formales. El uso de un logger permite gestionar niveles de información (INFO, ERROR, etc.) y mejorar el mantenimiento. Esta mejora contribuye a una arquitectura más profesional y preparada para producción.

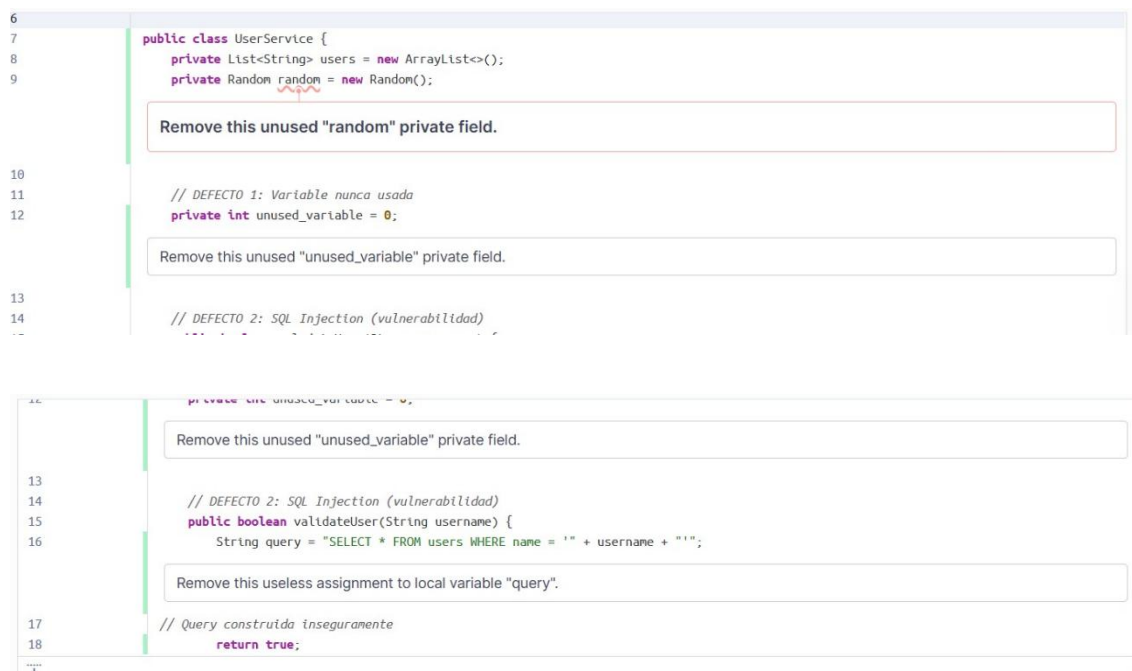
Anti-patrón:

Uso de consola para logging y mal uso de printf.

Patrón aplicado (Dos alternativas):

- Uso de Logger (Best Practice de monitoreo).
- Alternativamente, integración con sistemas externos de monitoreo, pero no aplica en este contexto simple.

MEDIUM 7



El código presenta múltiples problemas: variables no utilizadas, código innecesario y una vulnerabilidad crítica de SQL Injection. Construir consultas concatenando strings es inseguro. La solución usa Prepared Statements, lo que protege contra ataques y mejora la seguridad. Además,

se limpia el código eliminando elementos innecesarios, logrando una implementación más segura, clara y profesional.

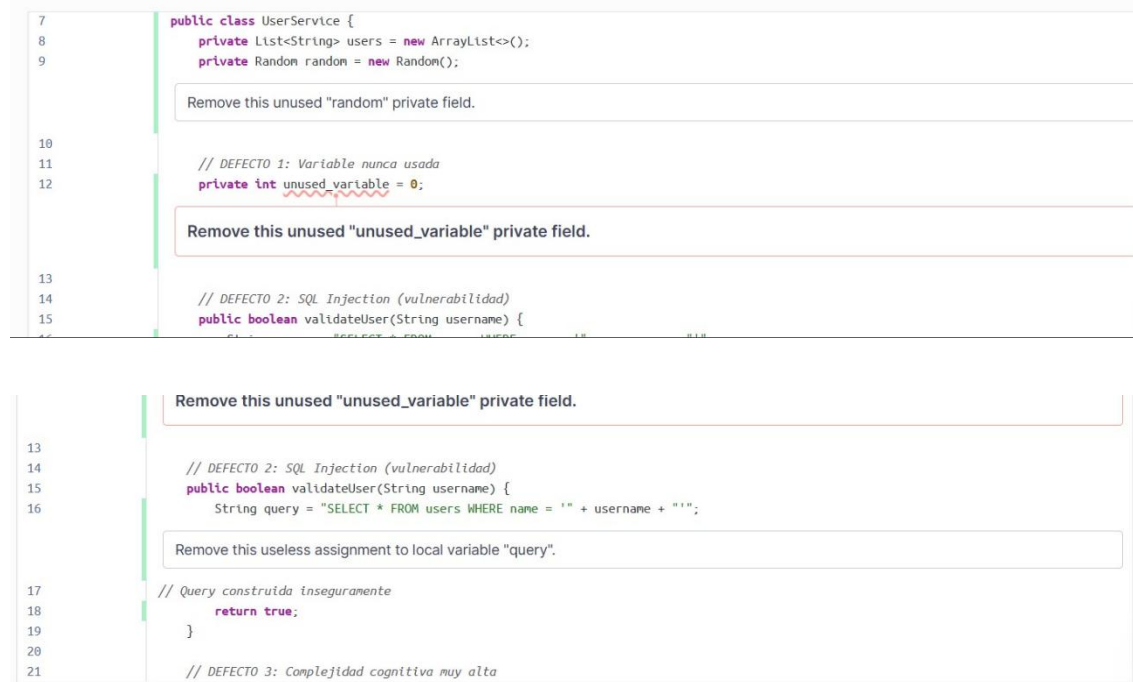
Anti-patrón:

Variables no utilizadas, vulnerabilidad de SQL Injection y código innecesario.

Patrón aplicado (Dos alternativas):

- Se aplica Prepared Statements (Security Pattern) y limpieza de código.
- También podría aplicarse el patrón Repository, encapsulando acceso a datos, pero aquí basta con asegurar la consulta.

MEDIUM 8



Se repiten problemas de código muerto y vulnerabilidad SQL. Esto demuestra falta de buenas prácticas en el manejo de datos. La solución elimina variables innecesarias y aplica

consultas parametrizadas, mejorando la seguridad del sistema. El resultado es un código más limpio, eficiente y alineado con estándares de desarrollo seguro.

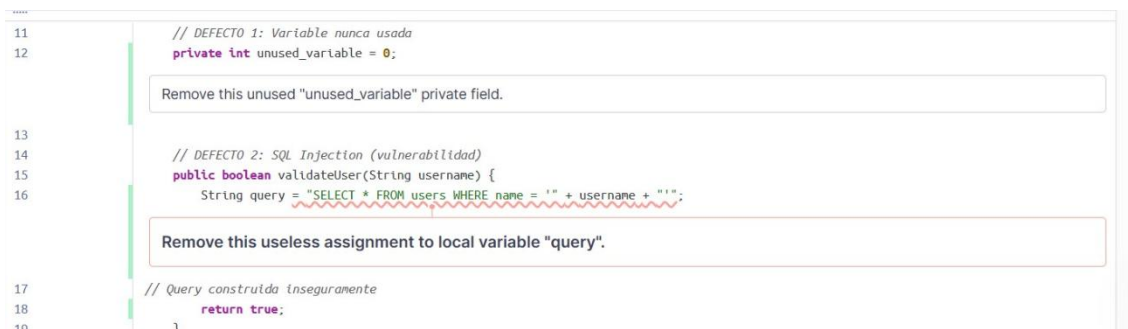
Anti-patrón:

Código muerto, variables innecesarias y vulnerabilidad SQL.

Patrón aplicado (Dos alternativas):

- Uso de Clean Code + Prepared Statements.
- Alternativamente, aplicar DAO Pattern, pero sería sobrediseño en este caso.

MEDIUM 9



El código presenta alta complejidad cognitiva debido a múltiples condiciones anidadas. Esto dificulta su comprensión y mantenimiento. La solución simplifica la lógica usando validaciones claras y reduciendo la anidación. Esto mejora la legibilidad, facilita futuras modificaciones y reduce errores, manteniendo la funcionalidad original.

Anti-patrón:

Alta complejidad cognitiva por anidación excesiva.}

Patrón aplicado (Dos alternativas):

- Aplicación de Guard Clauses y simplificación de condiciones.
- Alternativamente, uso de Strategy Pattern, pero innecesario por simplicidad.

MEDIUM 10



Se identifica el anti-patrón Arrow Code, donde la anidación excesiva hace el código difícil de leer. La solución aplica Guard Clauses, reduciendo niveles de profundidad y simplificando el flujo lógico. Esto permite un código más limpio, fácil de entender y mantener, cumpliendo además con límites de complejidad cognitiva.

Anti-patrón:

Arrow Code (anidación profunda).

Patrón aplicado (Dos alternativas):

- Uso de Guard Clauses.
- También podría aplicarse Strategy Pattern, pero no es necesario.

MEDIUM 11

```
40         return "UNKNOWN";  
41     }  
42  
43     // DEFECTO 4: Excepción no capturada  
44     public void addUser(String username) {  
45         int id = Integer.parseInt(username); // ¿Y si username no es un número?  
46  
47         users.add(username);  
48     }  
49  
50     // DEFECTO 5: Método nunca usado  
51     public int calculateUnusedMetric(int a, int b) {  
52         return a + b;  
53     }
```

Remove this useless assignment to local variable "id".

El código contiene una excepción no controlada y un método innecesario. Esto puede provocar fallos en ejecución y aumenta la complejidad del sistema. La solución implementa manejo de excepciones adecuado y elimina código no utilizado. Esto mejora la estabilidad, claridad y mantenimiento del sistema.

Anti-patrón:

Excepción no controlada y método sin uso.

Patrón aplicado (Dos alternativas):

- Uso de Exception Handling Pattern y eliminación de código muerto.
- Alternativamente, validación previa con Guard Clauses.

MEDIUM 12

```
50 public int calculateUnusedMetric(int a, int b) {
51     return a + b;
52 }
53
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception {
56     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
57         .forEach(System.out::println);
58     return "File read"; // Stream no se cierra explícitamente
59 }
60
61 public List<String> getUsers() {
62     return users;
63 }
64 }
```

Replace generic exceptions with specific library exceptions or a custom exception.

Replace this use of System.out by a logger.

Replace this use of System.out by a logger.

Se presenta un problema de fuga de recursos al no cerrar correctamente un stream, además del uso de excepciones genéricas y salida por consola. La solución utiliza try-with-resources, asegurando el cierre automático de recursos. También mejora el manejo de errores y logging, logrando un código más seguro, eficiente y profesional.

Anti-patrón:

Resource leak, uso de excepción genérica y System.out.

Patrón aplicado (Dos alternativas):

- Uso de Try-With-Resources.
- También podría aplicarse Facade Pattern para manejo de archivos, pero es innecesario aquí.

MEDIUM 13

```
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception {
    Replace generic exceptions with specific library exceptions or a custom exception.

    java.nio.file.Files.lines(java.nio.file.Paths.get(path))
    .forEach(System.out::println);
    Replace this use of System.out by a logger.

    return "File read"; // Stream no se cierra explícitamente
}

60 public List<String> getUsers() {
61     return users;
62 }
```

El código repite problemas de manejo de recursos, excepciones genéricas y logging inadecuado. Esto puede generar fallos y dificultar el mantenimiento. La solución aplica try-with-resources y uso de logger, mejorando la gestión de recursos y la trazabilidad. El resultado es un código más robusto, limpio y alineado con buenas prácticas.

Anti-patrón:

Recurso no cerrado + excepción genérica + logging incorrecto.

Patrón aplicado (Dos alternativas):

- Aplicación de Try-With-Resources + Logger.
- Alternativamente, uso de Service Layer Pattern para encapsular lectura de archivos, pero no es necesario en este caso.

ISSUE LOW

Low 1



Patrón aplicado

- Encapsulación (Information Hiding)

Defecto 2: El método deposit presenta una validación incorrecta al permitir valores negativos o no manejar adecuadamente este caso. Esto puede generar inconsistencias en el saldo de la cuenta, afectando la lógica del sistema. La ausencia de una validación clara permite operaciones inválidas que comprometen la integridad de los datos.

Anti-patrón

Lógica defectuosa / Validación incorrecta

Patrón aplicado

- Guard Clauses (validación temprana)

Código corregido:

```
public void deposit(double amount) {  
  
    if (amount <= 0) {  
  
        throw new IllegalArgumentException("El monto debe ser mayor que cero");  
  
    }  
  
    balance += amount;  
  
}
```

Low 2

```
public class BankAccount {  
    private double balance;  
    private String accountNumber;  
  
    // DEFECTO 7: Campo mutable sin sincronización  
    public double balance_public = 0.0;
```

Make balance_public a static final constant or non-public and provide accessors if needed.

Rename this field "balance_public" to match the regular expression `^[a-z][a-zA-Z0-9]*$`.

El atributo `balance_public` no cumple con las convenciones de nomenclatura de Java, ya que utiliza `snake_case` en lugar de `camelCase`. Según los estándares, los nombres de variables deben iniciar en minúscula y continuar con letras o números sin guiones bajos. Esto afecta la legibilidad y consistencia del código, dificultando su mantenimiento en equipos de desarrollo.

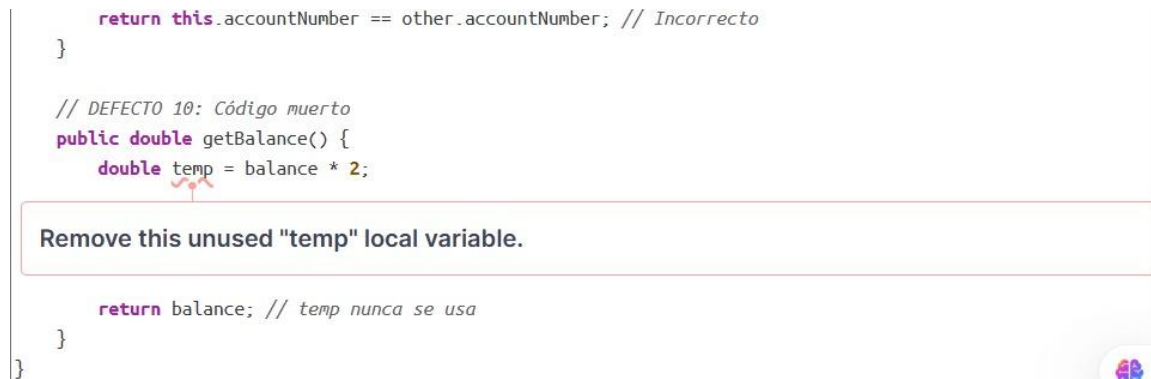
Anti-patrón

Bad Naming (Convenciones de nombres incorrectas)

Patrón aplicado

- Naming Conventions (Clean Code)

low 3



En el método getBalance, se declara una variable local temp que nunca es utilizada. Esto constituye código muerto, ya que no aporta ninguna funcionalidad al programa. La presencia de variables innecesarias afecta la claridad del código, puede confundir a otros desarrolladores y dificulta el mantenimiento. Además, incrementa innecesariamente la complejidad del método.

Anti-patrón

Dead Code (Código muerto)

Patrón aplicado

- Clean Code (eliminación de código innecesario)

Código corregido:

```
public double getBalance() {  
  
    return balance;  
  
}
```

Low 4



```
// DEFECTO 1: Variable nunca usada  
private int unused_variable = 0;
```

Rename this field "unused_variable" to match the regular expression `^[a-z][a-zA-Z0-9]*$`.

```
// DEFECTO 2: SQL Injection (vulnerabilidad)  
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = '" + username + "'";
```

Remove this unused "query" local variable.

El atributo `unused_variable` no cumple con las convenciones de nomenclatura de Java, ya que utiliza `snake_case` en lugar de `camelCase`. Según los estándares, los nombres de variables deben iniciar en minúscula y continuar con letras o números sin guiones bajos. Esto afecta la legibilidad, consistencia y mantenibilidad del código dentro del proyecto.

Anti-patrón

Bad Naming (Convenciones de nombres incorrectas)

Patrón aplicado

- Naming Conventions (Clean Code)

low 5



En el método validateUser, se declara la variable query, pero nunca se utiliza. Esto constituye código muerto, ya que no aporta ninguna funcionalidad. Además, la consulta está construida de forma insegura mediante concatenación de strings, lo que podría generar vulnerabilidades como SQL Injection si se utilizara. La presencia de código innecesario afecta la claridad y mantenibilidad del sistema.

Anti-patrón

Dead Code (Código muerto)

(Relacionado) Construcción insegura de consultas

Patrón aplicado

- Clean Code (eliminación de código innecesario)
- (Recomendado) Uso de consultas parametrizadas

Low 6

```
11 // DEFECTO 1: Variable nunca usada
12 private int unused_variable = 0;

13
14 // DEFECTO 2: SQL Injection (vulnerabilidad)
15 public boolean validateUser(String username) {
16     String query = "SELECT * FROM users WHERE name = '" + username + "'";
17
18     // Query construida inseguramente
19     return true;
}
```

Rename this field "unused_variable" to match the regular expression "[a-z][a-zA-Z0-9]*\$".

Remove this unused "query" local variable.

En el método validateUser, se declara la variable query, pero no se utiliza en ninguna operación posterior. Esto constituye código muerto, ya que no aporta funcionalidad al sistema. Además, la consulta está construida mediante concatenación de cadenas, lo que representa un riesgo potencial de seguridad (SQL Injection). La presencia de este código innecesario afecta la claridad, mantenibilidad y calidad del código.

Anti-patrón

Dead Code (Código muerto)

Patrón aplicado

- Clean Code (eliminación de código innecesario)
- (Adicional) Validación simple sin uso de SQL

Low 7

```
public String processUser(String user) {  
    if (user != null) {  
        if (user.length() > 0) {  
  
            if (user.contains("admin")) {  
                if (user.contains("test")) {  
                    if (user.contains("prod")) {  
                        return "ADMIN_TEST_PROD";  
                    } else {  
                        return "ADMIN_TEST";  
                    }  
                } else {  
                    return "ADMIN";  
                }  
            }  
        }  
    }  
}
```

Use "isEmpty()" to check whether a "String" is empty or not.

En el método processUser, se utiliza `user.length() > 0` para validar si un String está vacío. Aunque funcional, esta práctica no es la recomendada en Java, ya que reduce la legibilidad del código. El uso de `isEmpty()` es más claro, expresivo y alineado con las buenas prácticas. Este tipo de detalle afecta la calidad del código y su mantenibilidad, especialmente en equipos de desarrollo.

Anti-patrón

Bad Practice / Poor Readability

Patrón aplicado

- Clean Code (uso de métodos expresivos)



Low 8

```
0         return "UNKNOWN";
1     }
2
3     // DEFECTO 4: Excepción no capturada
4     public void addUser(String username) {
5         int id = Integer.parseInt(username); // ¿Y si username no es un número?
6
7         users.add(username);
8     }
9
10    // DEFECTO 5: Método nunca usado
11    public int calculateUnusedMetric(int a, int b) {
12        return a + b;
13    }
```

Remove this unused "id" local variable.

En el método `addUser`, se declara la variable `id` mediante `Integer.parseInt(username)`, pero nunca se utiliza posteriormente. Esto constituye código muerto, ya que no aporta funcionalidad al sistema. Además, esta conversión puede lanzar una excepción si el valor no es numérico, lo que introduce un riesgo innecesario. La presencia de variables sin uso afecta la claridad, mantenibilidad y calidad del código.

Anti-patrón

Dead Code (Código muerto)

Patrón aplicado

- Clean Code (eliminación de código innecesario)



Antes BankAccount

```
BankAccount.java x
1 package com.corhuila.calidad;
2
3 public class BankAccount { 1 usage
4     private double balance; 5 usages
5     private String accountNumber; 3 usages
6
7     // DEFECTO 7: Campo mutable sin sincronización
8     public double balance_public = 0.0; no usages
9
10    public BankAccount(String accountNumber, double initialBalance) { no usages
11        this.accountNumber = accountNumber;
12        this.balance = initialBalance;
13    }
14
15    // DEFECTO 8: Lógica ilógica
16    public void deposit(double amount) { no usages
17        if (amount < 0) {
18            System.out.println("Error: cantidad negativa");
19            balance = balance + amount; // ¡Sigue depositando cantidad negativa!
20        }
21    }
22
23    // DEFECTO 9: Comparación con == en lugar de equals()
24    public boolean isSameAccount(BankAccount other) { no usages
25        return this.accountNumber == other.accountNumber; // Incorrecto
26    }
27
28    // DEFECTO 10: Código muerto
29    public double getBalance() { no usages
30        double temp = balance * 2;
31        return balance; // temp nunca se usa
32    }
33 }
```



Después BankAccount, con las correcciones implementadas en el código

```
BankAccount.java x
1 package com.corhuila.calidad;
2
3 public class BankAccount { 1 usage
4
5     private double balance; 3 usages
6     private final String accountNumber; // final: el número de cuenta no debería cambiar 3 usages
7
8     // CORRECCIÓN 7: Eliminado campo público mutable.
9     // Exponer estado interno directamente rompe el encapsulamiento
10
11     public BankAccount(String accountNumber, double initialBalance) { no usages
12         if (accountNumber == null || accountNumber.isBlank()) {
13             throw new IllegalArgumentException("El número de cuenta no puede ser vacío.");
14         }
15         if (initialBalance < 0) {
16             throw new IllegalArgumentException("El saldo inicial no puede ser negativo.");
17         }
18         this.accountNumber = accountNumber;
19         this.balance = initialBalance;
20     }
21
22     // CORRECCIÓN 8: La lógica ahora es coherente.
23     // Si el monto es inválido, se lanza excepción y NO se modifica el saldo.
24     public void deposit(double amount) { no usages
25         if (amount <= 0) {
26             throw new IllegalArgumentException("El monto a depositar debe ser mayor a cero.");
27         }
28         balance += amount;
29     }
30
31     // CORRECCIÓN 9: Comparación de Strings con equals(), no con ==.
32     // == compara referencias en memoria, equals() compara el contenido.
33     public boolean isSameAccount(BankAccount other) { no usages
34         if (other == null) return false;
35         return this.accountNumber.equals(other.accountNumber);
36     }
37
38     // CORRECCIÓN 10: Eliminada variable 'temp' que nunca se usaba (código muerto).
39     public double getBalance() { no usages
40         return balance;
41     }
42 }
```



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Antes Main

```
© Main.java x
1 package com.corhuila.calidad;
2
   To Run code, press Ctrl F5 or click the ▶ icon in the gutter.
5 public class Main { no usages
6
7 }
```

Después Main

```
© BankAccount.java  © Main.java x
1 package com.corhuila.calidad;
2
   To Run code, press Ctrl F5 or click the ▶ icon in the gutter.
5 ▶ public class Main {
6 ▶     public static void main(String[] args) {
7         // Método principal vacío intencionalmente.
8         // Este proyecto no requiere ejecución directa.
9     }
10 }
```



Antes UserService

```
UserService.java x
1 package com.corhuila.calidad;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Random;
6
7 public class UserService { 6 usages
8     private List<String> users = new ArrayList<>(); 2 usages
9     private Random random = new Random(); no usages
10
11     // DEFECTO 1: Variable nunca usada
12     private int unused_variable = 0; no usages
13
14     // DEFECTO 2: SQL Injection (vulnerabilidad)
15     public boolean validateUser(String username) { 1 usage
16         String query = "SELECT * FROM users WHERE name = '" + username + "'";
17         // Query construida inseguramente
18         return true;
19     }
20
21     // DEFECTO 3: Complejidad cognitiva muy alta
22     public String processUser(String user) { 1 usage
23         if (user != null) {
24             if (user.length() > 0) {
25                 if (user.contains("admin")) {
26                     if (user.contains("test")) {
27                         if (user.contains("prod")) {
28                             return "ADMIN_TEST_PROD";
29                         } else {
30                             return "ADMIN_TEST";
31                         }
32                     }
33                 }
34             }
35         }
36     }
37 }
```




CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

```
32         } else {
33             return "ADMIN";
34         }
35     } else {
36         return "USER";
37     }
38 }
39 }
40 return "UNKNOWN";
41 }
42
43 // DEFECTO 4: Excepción no capturada
44 public void addUser(String username) { 1 usage
45     int id = Integer.parseInt(username); // ¿Y si username no es un número?
46     users.add(username);
47 }
48
49 // DEFECTO 5: Método nunca usado
50 > public int calculateUnusedMetric(int a, int b) { return a + b; }
51
52
53
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception { no usages
56     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
57         .forEach(System.out::println);
58     return "File read"; // Stream no se cierra explícitamente
59 }
60
61 > public List<String> getUsers() { return users; }
62 }
```

Después UserService, con las correcciones implementadas en el código.

```
BankAccount.java Main.java UserService.java x
10
11 public class UserService { 7 usages
12
13     private static final Logger logger = Logger.getLogger(UserService.class.getName()); 1 usage
14
15     private final List<String> users = new ArrayList<>(); 2 usages
16
17     public boolean validateUser(String username) { 1 usage
18         if (username == null || !username.matches(regex: "[a-zA-Z0-9_]*")) {
19             return false;
20         }
21         return true;
22     }
23
24     public String processUser(String user) { 1 usage
25         if (user == null || user.isEmpty()) {
26             return "UNKNOWN";
27         }
28
29         boolean isAdmin = user.contains("admin");
30         boolean isTest = user.contains("test");
31         boolean isProd = user.contains("prod");
32
33         if (isAdmin && isTest && isProd) return "ADMIN_TEST_PROD";
34         if (isAdmin && isTest) return "ADMIN_TEST";
35         if (isAdmin) return "ADMIN";
36
37         return "USER";
38     }
39
40     public void addUser(String username) { 1 usage
41         if (username == null || username.isBlank()) {
42             throw new IllegalArgumentException("El username no puede ser nulo o vacío.");
43         }
44         try {
45             Integer.parseInt(username);
46         } catch (NumberFormatException e) {
47             throw new IllegalArgumentException("El username debe ser numérico: " + username, e);
48         }
49         users.add(username);
50     }
51 }
```



```
public void addUser(String username) { 1 usage
    if (username == null || username.isBlank()) {
        throw new IllegalArgumentException("El username no puede ser nulo o vacío.");
    }
    try {
        Integer.parseInt(username);
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("El username debe ser numérico: " + username, e);
    }
    users.add(username);
}

// IOException en lugar de Exception genérico
public String readUserFile(String path) throws IOException { no usages
    try (var lines = Files.lines(Paths.get(path))) {
        // Logger en lugar de System.out
        lines.forEach( String line -> logger.info(line));
    }
    return "File read";
}

> public List<String> getUsers() { return Collections.unmodifiableList(users); }
}
```

Conclusiones

- Evidenciamos una falta de la aplicación de buenas prácticas de codificación, lo que generaba problemas como código poco legible, difícil de mantener y con estructuras innecesariamente complejas. Esto afectando directamente la calidad del software.
- Presentaba una alta complejidad cognitiva, especialmente por el uso de anidaciones profundas (Arrow Code), lo que requería la necesidad de aplicar técnicas de refactorización como Guard Clauses, que simplifican el flujo lógico y mejoran la comprensión del código.
- Presenciamos deficiencias importantes en términos de seguridad, como la vulnerabilidad a SQL Injection, lo que indica que no se están aplicando prácticas seguras en el manejo de datos. El uso de consultas parametrizadas es fundamental para evitar estos riesgos



- Se encontraron errores lógico-semánticos los cuales son peligrosos porque el código compila correctamente, pero se comporta de forma inesperada en tiempo de ejecución. No los detecta el compilador sino herramientas de análisis estático como SonarQube o pruebas unitarias.
- Al no aplicar programación defensiva. Al no validar entradas del usuario se generan vulnerabilidades como SQL Injection y excepciones no controladas que comprometen la estabilidad y seguridad del sistema.
- Defectos de mantenibilidad: No rompen la funcionalidad del programa, pero incrementan la deuda técnica dificultando la lectura, modificación y escalabilidad del código a largo plazo.